Impactis Project

Third Milestone Report — Asynchronous Order Processing with Kafka

1. Summary

This milestone makes **POST /order** non-blocking. Previously the endpoint performed every step inside the HTTP request: validating the user, restaurant and items, pricing the order, and writing it to PostgreSQL. It now saves the order as **PENDING**, publishes it to an Apache Kafka topic, and returns **202 Accepted** immediately. A separate consumer performs the validation and pricing in the background and moves the order to **CONFIRMED** or **FAILED**. The four design considerations set for this milestone were addressed and each was checked with a real test against a running Kafka broker: partition count, no lost messages (retries and a dead letter topic), no duplicate processing, and horizontal consumer scaling.

Under a k6 load test ramping to 120 orders per second, the API accepted 2,699 orders with an average accept time of 20 ms (p95 34 ms) and no failed HTTP requests. The number of orders confirmed in the database rose by exactly 2,699, and the consumer group finished with zero lag on all partitions.

2. Design Decisions

The decisions below were made before implementation. Each is the common industry choice for a system of this size.

Question	Decision	Reason
Partitions	3, keyed by restaurantId	Gives three-way parallelism. Messages with the same key always go to the same partition, so one restaurant's orders are processed in the order they were placed.
What does POST /order return?	202 Accepted, a Location header, and the order with status PENDING	201 would imply the order is complete. 202 says the request was accepted but is not finished, and the Location header points to GET /order/{id} for checking progress.
Is the order row created up front?	Yes, as PENDING, after checking the user and restaurant exist	The client gets a real order ID at once, and the row always points to a real user and restaurant, so the database foreign keys stay strict. Only the cheap existence checks run in the request.
Business failure vs. system failure	Business failures set the order to FAILED. Unexpected errors are retried, then dead-lettered	A closed restaurant is a normal outcome, not a fault. Only genuine processing errors belong in the dead letter topic.
Retry policy	Exponential backoff: 1 s, doubling, up to 7 s total	Waiting longer between attempts gives a struggling dependency time to recover. The total is short enough to observe in a test.

Duplicate protection	Consumer skips any order that is no longer PENDING	Kafka can redeliver a message after a crash. Re-running the work on an order that is already processed is made a no-op.
Scaling	One consumer group, 3 consumer threads per instance	Kafka divides partitions among group members automatically. More consumers than partitions would sit idle.

3. Order Flow

Request path (synchronous, fast):

- Request body validated (userId, restaurantId and at least one valid item are required).
- User and restaurant looked up by ID. If either is missing, the request fails with 404.
- Order saved with status PENDING and a newly generated idempotency key.
- An OrderEvent is published to the `order-events` topic, keyed by restaurantId.
- Response: 202 Accepted with a Location header and the PENDING order.

Background path (asynchronous):

- The consumer receives the event and skips it if the order is no longer PENDING.
- It checks that the user is ACTIVE, the restaurant is active, every menu item is available, and every menu item belongs to the restaurant on the order.
- It looks up real prices, calculates subtotal, delivery fee and total, saves the order items, and sets the order to CONFIRMED, all in one transaction.
- If any business check fails, the order is set to FAILED with a human-readable failureReason.
- If an unexpected error occurs, the message is retried with backoff and then moved to the dead letter topic.

Prices are always looked up from the database by the consumer; the client never supplies them. Clients follow progress with **GET /order/{id}**, which returns PENDING, CONFIRMED (with totals and items) or FAILED (with failureReason).

4. Implementation

4.1 Infrastructure

- Zookeeper and Kafka (Confluent 7.6.0 images) were added to Docker Compose with healthchecks, alongside PostgreSQL and Redis.
- Kafka has two listeners. Other containers reach it at kafka:9092, while the application, which runs on the host machine, uses localhost:29092. Both ports are published.
- The Kafka connection is configured through spring.kafka.* properties, with the bootstrap server overridable by environment variable like the other connections.

4.2 Topics

Topic	Partitions	Replicas	Purpose
order-events	3	1	Order events, keyed by restaurantId
order-events.DLT	3	1	Messages that failed every retry

Replication factor 1 is the only valid value with a single local broker. A production deployment would run several brokers with a replication factor of 3.

4.3 Producer and Order Status Endpoint

OrderService now only validates existence, saves the PENDING order, and publishes the event. OrderController returns 202 with a Location header and exposes the new GET /order/{id}. The producer runs with acknowledgements from all replicas (acks=all) and idempotent publishing enabled.

4.4 Consumer

OrderEventConsumer holds the business logic that used to run inside the request. It is a @KafkaListener in the consumer group `order-processing-group`, runs three threads per instance to match the partition count, and processes each message in a database transaction.

4.5 Retry and Dead Letter Topic

A DefaultErrorHandler retries a failing message with exponential backoff (1 s, then 2 s, then 4 s). When the 7-second budget is spent, a DeadLetterPublishingRecoverer publishes the message to `order-events.DLT` on the same partition number.

A dead-lettered message is safe but, on its own, leaves its order PENDING forever and the client cannot tell it failed. A second listener, OrderDeadLetterConsumer, closes that gap. It runs in its own group (`order-dlt-group`) and marks the order FAILED with a deliberately generic reason, so internal error details are never returned to clients. It only changes orders that are still PENDING, and it never throws: the shared error handler routes failures into this same topic, so an exception here would loop.

4.6 Database Migrations (Flyway)

Version	Change

V2	Added orders.idempotency_key (UUID, unique)
V3	Added orders.failure_reason (VARCHAR 500)
V4	Added a BEFORE UPDATE trigger on all five tables so updated_at changes on every update
V5	Corrected the status of legacy orders created before Kafka (see section 6)

5. Verification of the Design Considerations

5.1 Non-blocking behaviour

An order placed through curl returned **202** with a Location header and status PENDING, with empty items and null totals. Fetching the order moments later showed CONFIRMED, with correct pricing (16.46 × 2 = 32.92 subtotal, plus a 1.98 delivery fee, giving a 34.90 total) and the menu item name resolved server-side.

5.2 Messages are never processed twice

A duplicate delivery was simulated by publishing a second message for an already CONFIRMED order directly with the Kafka console producer. The consumer received it, logged that the order was already processed, and skipped it. Fetching the order afterwards showed identical data: one item, same total.

Note: the duplicate check is based on the order's status. The idempotency key is stored on the order and carried in the message, but the consumer does not currently compare keys.

5.3 Messages are never lost

To force a genuine system failure, a deliberate exception was temporarily placed in the consumer, then removed after the test. The log showed four delivery attempts (the original plus three retries) spaced about 1, 2 and 4 seconds apart, matching the backoff configuration. The identical message, with the same order ID and payload, was then read back from the `order-events.DLT` topic.

The order from that test stayed PENDING until the dead letter listener was added. On the next start, the listener consumed the stored message and the order showed FAILED with the generic reason, confirming the whole path end to end.

5.4 Horizontal scaling

Threads within one instance. With concurrency set to 3, three orders placed for three different restaurants were handled by three different consumer threads, each owning one partition.

Separate instances. A second copy of the application was started on another port. Both processes joined the same consumer group, and Kafka redistributed the three partitions between them without any code change or manual step. In the final assignment recorded in the logs, one instance held partitions 1 and 2 and the other held partition 0. Because the two instances were started within seconds of each other, the group rebalanced twice before settling, which is normal.

Each instance runs three consumer threads, so two instances create six consumers for three partitions; three of them sit idle. Useful parallelism is capped by the partition count, which is why the partition count matters when planning for growth.

6. Supporting Improvements

- **Menu item ownership check.** A test order mixing a restaurant with a menu item from a different restaurant was initially accepted. The consumer now rejects it with FAILED and a clear reason, verified with a repeat order.
- **failureReason exposed.** The order response now includes the reason an order failed.

- **Reliable updated_at.** The schema's default timestamp only applied at insert, so confirmed orders showed an updated time equal to their created time. The V4 triggers fix this: a new order showed a confirmation about 223 ms after creation.
- **Correct 4xx responses.** Client mistakes that were returning 500 now return the proper status: a malformed ID, missing parameter or malformed JSON returns 400; an unknown path returns 404; a wrong HTTP method returns 405 with an Allow header; a wrong content type returns 415. Responses stay generic and do not echo internals.
- **Legacy order cleanup.** 150 orders from the first milestone had been left PENDING by the old synchronous code even though they had totals and items. Before changing anything, the data was checked: all 150 had no idempotency key, a total, and items. V5 sets exactly that signature to CONFIRMED, so genuinely queued orders can never match. Afterwards no orders remained PENDING, so a PENDING order now reliably means one still in flight.
- **API documentation.** The Swagger page now documents the 202 flow, the possible status values, and the status endpoint. A full browser run of POST /order followed by GET /order/{id} showed PENDING moving to CONFIRMED.

7. Automated Tests

The old order tests no longer matched the new design and were rewritten, and the new classes were covered:

- **OrderService:** saves a PENDING order, publishes to the right topic with restaurantId as the key, carries the same idempotency key in the message and on the order, and publishes nothing when the user or restaurant is missing.
- **OrderEventConsumer:** correct totals, default delivery fee, skip on duplicate or missing order, and a FAILED result for each of the inactive or missing user, inactive or missing restaurant, unavailable or missing item, and item from another restaurant.
- **OrderDeadLetterConsumer:** marks PENDING orders FAILED, leaves processed orders untouched, ignores bad records, and does not throw when the database fails.
- **OrderController:** 202 with a Location header, validation errors, the status endpoint, and every 4xx case above.

Result: 80 tests passing and the JaCoCo gate met (at least 90% line coverage). The coverage gate failed once during the milestone, at 89%, after a change that added no executable code, which showed there was little headroom. It was fixed properly by adding tests for resilience logic that had none: the Redis-down error handler, the cache metrics endpoint, and the generic 500 handler.

Retry, dead-lettering and rebalancing were verified manually against a real broker, as described in section 5, rather than re-implemented as automated tests with an embedded broker.

8. Load Test (k6)

The test sent orders at a steady 50 per second for 20 seconds, then ramped to 120 per second for 20 seconds. One order in ten was followed until it reached CONFIRMED or FAILED, to measure the time until processing finished. The database order counts were recorded before and after.

Measurement	Result
Orders accepted (202)	2,699
POST /order response time	avg 19.9 ms, median 18.0 ms, p95 34.0 ms, max 114 ms
Failed HTTP requests	0%
Time until CONFIRMED (265 sampled orders)	median 263 ms, p90 1.06 s, p95 2.05 s, max 3.29 s
CONFIRMED orders in database	2,705 after, 6 before: an increase of exactly 2,699
New FAILED orders	0
Consumer lag after the run	0 on all 3 partitions

Result: every order the API accepted was processed exactly once, with none lost or duplicated, while the accept time stayed around 20 ms during the burst. Processing time grew under the heavier load but the backlog fully cleared.

How to read these numbers: the 263 ms median for confirmation is mostly the test's own 250 ms polling interval, so the true median is lower. The 2 to 3 second tail is consistent with queueing during the peak, but per-stage figures were not captured, so that is an interpretation, not a measurement. The load was uneven: the final partition offsets (563, 536 and 1,609) show one

partition received roughly 60% of the messages. The test used only five restaurants and messages are keyed by restaurant, so a few restaurants map to the same partition. With many restaurants the load would spread more evenly, but a few very busy ones could still concentrate on one partition. This is the cost of per-restaurant ordering.

9. Integration Issues Encountered

As in earlier milestones, several problems came from running a very new stack (Spring Boot 4.1.1). They are recorded because they cost real time and may recur.

- **Zookeeper healthcheck.** The first healthcheck used netcat, which the image does not contain, and a four-letter command that newer Zookeeper versions disable. Zookeeper itself was healthy. The check was replaced with a plain TCP connection test.
- **Unreachable Kafka port.** The second (host) listener was configured but its port was not published, which would have blocked the application. Caught on review of the Compose file and fixed before it caused a failure.
- **Spring Boot 4 package move.** The Kafka properties class moved to a new package and needs the dedicated spring-boot-starter-kafka dependency; the method to build producer properties also changed signature.
- **Typed Kafka template.** Spring Boot provides a generic KafkaTemplate, which does not satisfy a constructor asking for a KafkaTemplate of String and OrderEvent. An explicitly typed bean was defined.
- **Migration ordering.** The new schema column was missing when Hibernate validated the schema, because Flyway ran as an ordinary bean and Spring's bean creation order shifted once Kafka was added. Flyway now runs at the very start of main(), before Spring starts, so the order can no longer vary.

10. Known Limitations

These are stated plainly so the results above are not over-read.

- **Publish failure after the database save (not tested).** The order row is saved first and the message published second, with no shared transaction. If Kafka is unreachable at publish time, an order could remain PENDING with no message behind it. The standard remedy is the transactional outbox pattern. The guarantee of no lost messages in this report covers failures during processing (retries and the dead letter topic), not this case.
- **Single broker.** With one Kafka broker and replication factor 1, the broker itself is a single point of failure. The Kafka and Zookeeper containers also have no persistent volume in the development Compose file, so recreating them discards the topics.
- **Duplicate check is status-based.** It relies on the order's status and has not been tested under two consumers racing for the same message.
- **No like-for-like comparison with the old endpoint.** The synchronous version was replaced, so it was not measured under this same load. The test shows what the client now waits for (about 20 ms) and how long processing takes, not a direct before-and-after.
- **Blocking retries.** While a message is being retried, its partition's consumer thread waits. This is acceptable at this scale but would delay later messages on that partition during a long failure.
- **Skewed partition load** in the load test, explained in section 8.
- **Tests are not isolated.** The application-context test connects to the real local Kafka and PostgreSQL, so running tests touches development data.

- **Unused code.** InvalidOrderException and its 409 handler are now unused, since validation moved into the consumer.

11. Conclusion

The third milestone is complete. POST /order is non-blocking: it responds in about 20 ms with a 202 and a status endpoint, while a Kafka consumer does the real work in the background. Each design consideration was addressed and verified against a running broker: three partitions keyed by restaurant; retries with exponential backoff and a dead letter topic whose messages also update the order's status; a duplicate-delivery guard demonstrated with a simulated redelivery; and consumer scaling shown both across threads and across two separate application instances. A load test at up to 120 orders per second showed no lost orders and a fully drained queue. The remaining gaps, chiefly the publish-failure case and the single-broker setup, are documented above, with the transactional outbox pattern and a multi-broker cluster as the natural next steps.